pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Course 3 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The brief: the desk lead's note and the rules

The desk lead's fifteenth note

The vendor's extract for July to early October is in, with a cover note. Before it touches our history, prove it against the note and the market's own calendar, fix what is wrong with a decision you can defend, and put the fixes in one function so tomorrow's file is one call. Reconcile it to the source files with nothing unexplained. Then append it and give me the snapshot for 2 October, and for 30 September from the same command. Keep both days in our database and write me one page. Describe it: no view on any rate or spread.

The file's problems are planted and unnamed. The Treasury yields are public data; the spreads and the book are synthetic.

The Rules

1. Work in a copy: fixes go to "extract", never to "raw".
2. State every decision in a markdown cell, with "headline()" before and after the fix.
3. The market calendar is the Treasury file's own dates; never fill a value from a neighbour.
4. Describe: never forecast, rank, or recommend.
5. Restart and Run All before you hand in.

How It Is Checked

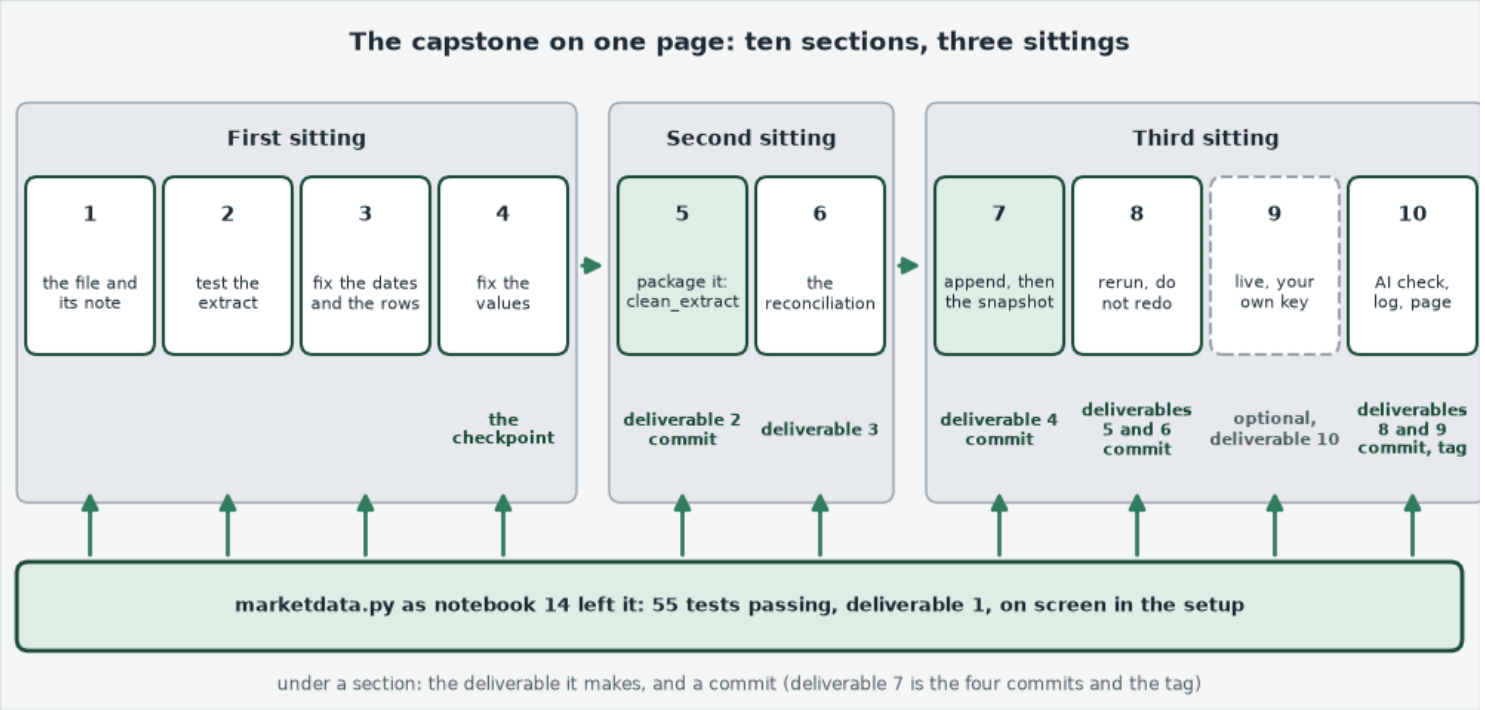
Each "check()" compares a fingerprint of your answer with a stored one; round as each question asks.

The checkpoint must pass before section 5.

A Stuck block holds the expected decision: open it after your own.

An AI assistant is allowed: the docstring is the prompt, and every answer is tested and logged in 10.2.

The capstone on one page



Ten sections, three sittings, one page at the end. Your module does the finding.

The data, the cover note, and the tests

Units: yields in percent, spreads in basis points, the book in dollars.

File	What it is
`course3_capstone_extract.csv`	the vendor's extract, read as text
`course3_capstone_cover_note.txt`	the vendor's note
`treasury_par_yields.csv`	the Treasury par curve
`course3_synthetic_spreads.csv`	two synthetic spreads
`holdings.csv`	the September book, synthetic

The clean history ends at "HISTORY_END", 2026-06-30. The Treasury file and the spreads run on past it: the course holds the extract's clean twin, so every decision can be proved.

The Tests Pass Before We Start

```
.....................................................   [100%]
55 passed in 0.50s
```

Deliverable 1 is on screen: 55 tests pass.

The Vendor's Cover Note, As It Came

```
To: Credit desk
From: Market data vendor, client operations
Subject: Daily extract, 2026-07-01 to 2026-10-02

The extract is attached as course3_capstone_extract.csv, in the usual
  layout.

Period:        2026-07-01 to 2026-10-02
Market days:   66
Columns:       date, 2 Yr, 5 Yr, 10 Yr, 30 Yr (par yields, percent),
  ig_spread_bp, hy_spread_bp (basis points)

Last values sent, for 2026-10-02:
  2 Yr     4.83 percent
  5 Yr     5.06 percent
  10 Yr    5.28 percent
  30 Yr    5.63 percent
  ig_spread_bp   97.0
  hy_spread_bp   328.7

The spread columns are synthetic course data, invented for this course.
  They are not ICE, Moody's, or any index.
Please tell us if the extract does not agree with these figures.
```

Setup: the files, headline() and check()

Imports and the Files

```
import subprocess
import sys
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
sns.set_theme(style='whitegrid', palette=['#2E7D5B', '#9AA5B1', '#1E4D3A', '#52606D'])
import marketdata as md
```

```
raw = pd.read_csv('course3_capstone_extract.csv', dtype=str)
options = {'parse_dates': ['date'], 'index_col': 'date', 'float_precision': 'round_trip'}
source = pd.read_csv('treasury_par_yields.csv', **options).join(
    pd.read_csv('course3_synthetic_spreads.csv', **options), validate='one_to_one')
september = pd.read_csv('holdings.csv')
```

Each "check()" compares a fingerprint of your answer with a stored one; round as each question asks.

The checkpoint must pass before section 5.

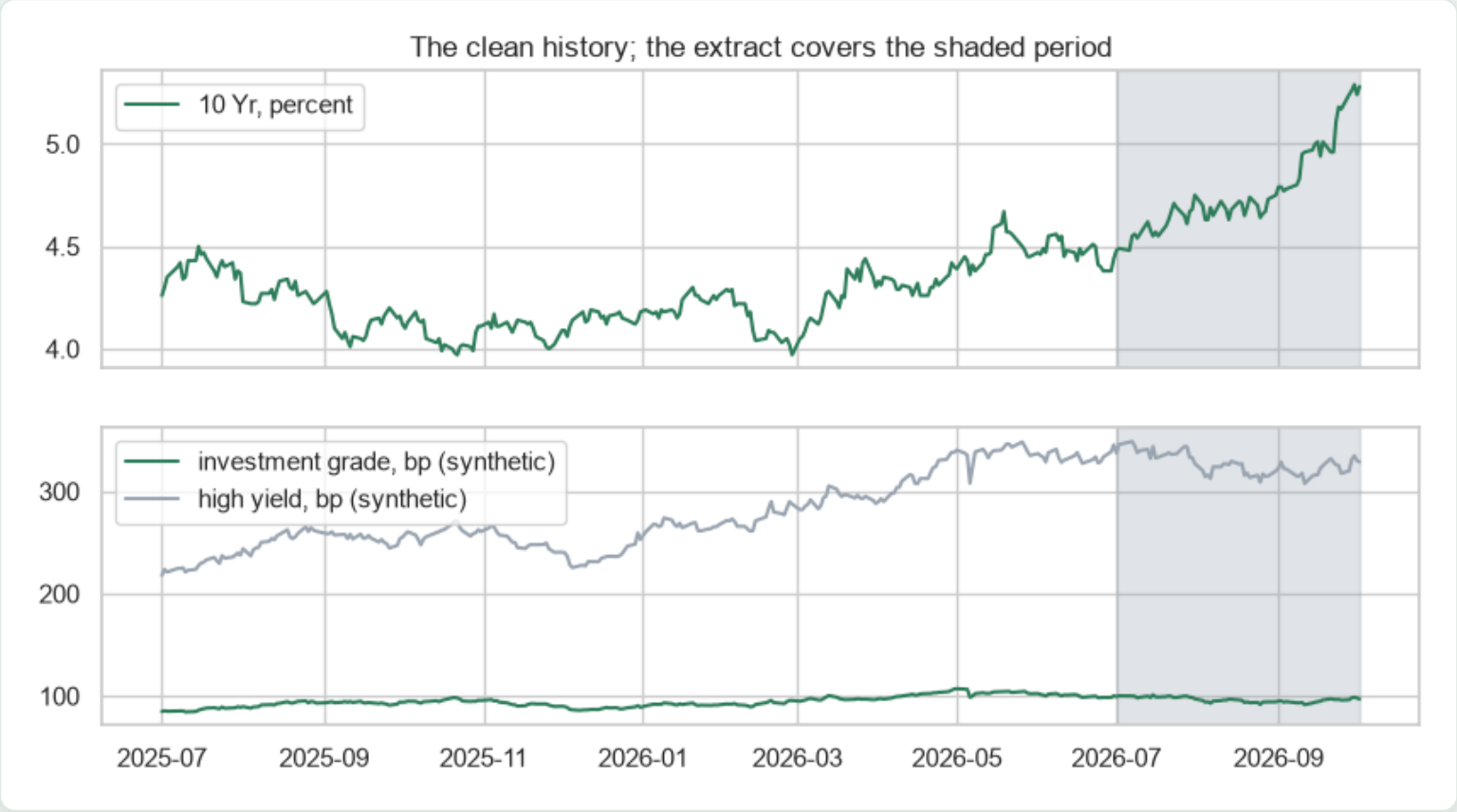
check()

```
import hashlib
def check(label, answer, code, places=None):
    if answer is ...:
        print(label + ': not attempted yet')
        return
    if places is not None:
        answer = round(float(answer), places)
    text = label + '|' + str(answer).strip().lower()
    if hashlib.sha256(text.encode()).hexdigest()[:12] == code:
        print(label + ': correct')
    else:
        print(label + ': not yet. You have ' + str(answer))
```

The Cover Note's Figures, and headline()

```
NOTE_DAYS = 66
NOTE_LAST = {'2 Yr': 4.83, '5 Yr': 5.06, '10 Yr': 5.28, '30 Yr': 5.63,
             'ig_spread_bp': 97.0, 'hy_spread_bp': 328.7}
VALUE_COLUMNS = list(NOTE_LAST)
YIELDS, SPREADS = VALUE_COLUMNS[:4], VALUE_COLUMNS[4:]
HISTORY_END = '2026-06-30'
def headline(label, table):
    empty = int((table == '.').sum().sum() + table.isna().sum().sum())
    print(f'{label}: {len(table)} rows, {empty} cells with no number')
```

Where the extract sits



Where the Extract Sits

Where does the extract's period fall in the clean history?

The clean history stops in June; the extract must earn its place after it.

1 The file and its note; 2 the seven rules

1 The File and Its Note

1.1 Rows, Columns, and a Working Copy

What shape is "raw" (answer_1_1)? Copy it to "extract".

1.2 The File Against Its Note

Print "headline()" as received. How many rows beyond the note's market days (answer_1_2a)? Are the columns in the note's order? How many of the last row's six values differ from the note (answer_1_2b)?

2 Test the Extract

Seven rules say what a clean extract looks like. Count on "extract" as 1.1 made it, and fix nothing until section 3.

Rule	What the desk expects
1	one row per market day, no row a copy
2	every date written YYYY-MM-DD
3	every date a market day
4	every market day a row
5	no value missing
6	the note's units: yields in percent, spreads in bp
7	no value stuck for 5 rows in a row

Tools: Course 1's "pd.to_datetime(errors='coerce')", "duplicated()" and "isin()".

2 Test the extract: four counts, and Excel

2.1 Rule 2: Dates

How many dates do not read as "YYYY-MM-DD" (answer_2_1)? Show them as written.

2.2 Rule 5: No Value Missing

How many cells hold "." (answer_2_2a), and in how many rows (answer_2_2b)? Show those rows.

2.3 Rule 1: Copies

How many rows are exact copies of an earlier row (answer_2_3)? Show each.

2.4 Rule 6: Units

Read the values as numbers, "." left out. How many yields are above 25 (answer_2_4a), and how many spreads below 10 (answer_2_4b)? Show them.

Excel Side by Side

Tested in desktop Excel for Windows.

Job	Python	Excel
A date read as YYYY-MM-DD	`format='%Y-%m-%d', errors='coerce'`	`=DATE(LEFT(A2,4),MID(A2,6,2),RIGHT(A2,2))`
. read as no value	`== '.'`	`=IFERROR(VALUE(B2),NA())`
A 5 Yr yield above 25	`> 25`	`=IFERROR(VALUE(C2),NA())>25`
A spread below 10	`< 10`	`=IFERROR(VALUE(F2),NA())<10, =IFERROR(VALUE(G2),NA())<10`

3 Fix the dates and the rows

Dates first: every other rule needs them. Tools: Course 1's ".fillna()" and "drop_duplicates()", notebook 03's "md.missing_weekdays".

3.1 Dates and Copies

Read every date in the form it was written, then remove the exact copies. How many rows are left (answer_3_1)?

3.2 The Row of Dots

One row holds "." in every value. Give its date as text year-month-day (answer_3_2a), and whether "source" has a row for it, "True" or "False" (answer_3_2b). Decide and fix.

3.3 Rules 3 and 4: The Market's Own Days

Test the dates against source's: give the date that is not a market day (answer_3_3a) and the market day with no row (answer_3_3b), as text. "md.missing_weekdays(...).intersection(source.index)" finds the second.

The market calendar is the Treasury file's own dates; never fill a value from a neighbour.

4 Fix the values, then the checkpoint

Tools: Course 1's ".loc[mask, column] =", notebook 03's "md.stale_runs" and "pd.concat".

4.1 Rule 6: Units

Set the dates as the index and read the values as floats. Fix the units by 2.4's thresholds. Give the largest yield (answer_4_1a) and the smallest spread (answer_4_1b) after the fix.

4.2 Rule 7: A Stale Run

Run "md.stale_runs(..., min_length=5)" on each column. Which column has a run (answer_4_2a), and on what date does it start, as text (answer_4_2b)?

4.3 Restore From the Source

Restore the repeats and the missing market day from "source", sort by date, and "assert" the dates equal the period's market days in "source". Give the restored day's 10 Yr (answer_4_3).

Checkpoint: The Cleaned Extract

Does "extract" match "source" and the expected decisions? Every line must read "correct" before section 5.

```
if extract is ...:
    print('Checkpoint: not attempted yet')
else:
    check('Checkpoint rows', len(extract), '4d3783af28ef', places=0)
    check('Checkpoint first date', f'{extract.index[0]:%Y-%m-%d}', 'bd218fac6b48')
    check('Checkpoint last date', f'{extract.index[-1]:%Y-%m-%d}', '9a51dd8ab64d')
    check('Checkpoint missing values', extract.isna().sum().sum(), '3117f3c606c8', places=0)
    gap = (extract - source.loc[extract.index, VALUE_COLUMNS]).abs().max().max()
    check('Checkpoint gap from source', gap, '8b80901df176', places=9)
    check('Checkpoint mean 5 Yr', extract['5 Yr'].mean(), '865b6d2992f1', places=3)
    check('Checkpoint mean hy', extract['hy_spread_bp'].mean(), '8ea3cb50eb1d', places=3)
    check('Checkpoint raw is unchanged', raw.shape, 'd0733f5fa2a6')
```

5 Package it: clean_extract

5.1 clean_extract, Docstring First

In "clean_extract.py", written with "%%writefile" in a cell of its own, write "clean_extract(raw, source)": a docstring listing your seven decisions, then the decisions in that order, its inputs unchanged. It returns "(cleaned, log)", "log" one row per change: "date", "column" ("all" for a whole row), "received", "cleaned", "decision". "assert" each rule it promises, with a message. Store its result on "raw" in cleaned and log, and "assert" the note's market days and last values.

As Received Against Cleaned

Written for you, and drawn once "cleaned" exists: the file's numbers as sent against "cleaned". A value far outside the cleaned range sits on its panel's edge.

As Received Against Cleaned: the Cell

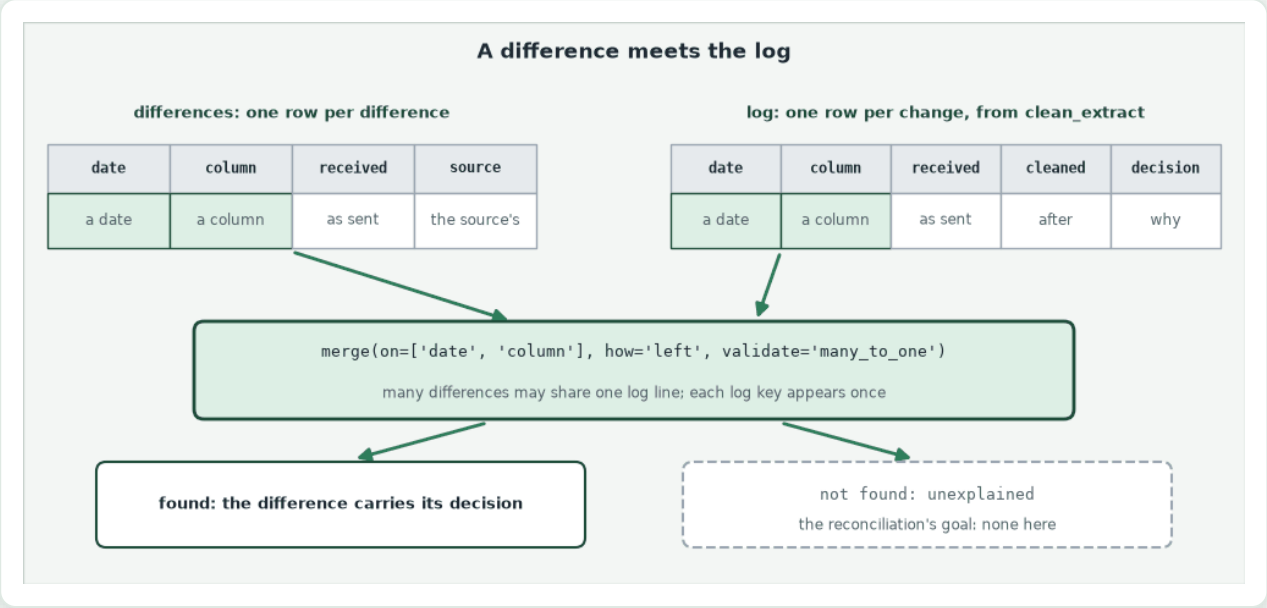
```
if cleaned is not ...:
    dates = pd.to_datetime(raw['date'], format='%Y-%m-%d', errors='coerce')
    fig, axes = plt.subplots(6, 1, figsize=(10, 9), sharex=True)
    for ax, column in zip(axes, VALUE_COLUMNS):
        low, high = cleaned[column].min() * 0.97, cleaned[column].max() * 1.03
        sent = pd.to_numeric(raw[column], errors='coerce').clip(low, high)
        ax.scatter(dates, sent, s=14, color='#9AA5B1')
        ax.plot(cleaned.index, cleaned[column], color='#2E7D5B')
        ax.set_ylabel(column)
    plt.show()
```

5.2 Commit It

Commit "clean_extract.py" alone, with a subject naming the deliverable, and show the history.

One call rebuilds the cleaned extract from the file as it arrived.

6 The reconciliation



A reconciliation sets the file as received beside the source files and gives every difference a decision from "log".

Every Difference by Its Decision

Written for you: which decision covers the most differences?

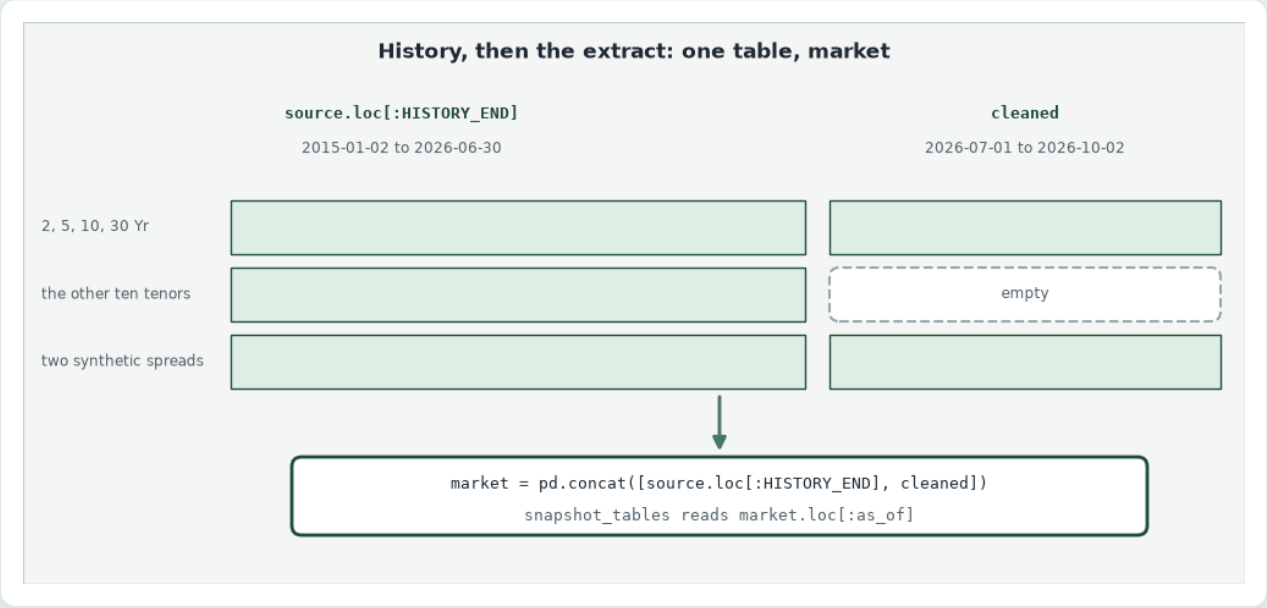
6.1 The File Against the Source

Read raw's dates in both forms and fix nothing else. Build differences ("date" as text, "column", "received", "source"): a row per extra copy, per row on a day with no market, per market day with no row (column "all"), and per value that differs from "source". How many (answer_6_1)?

6.2 A Decision for Every Difference

Merge "differences" with "log" on "date" and "column", "validate='many_to_one'", as explained. How many have no decision (answer_6_2a), and how many decisions do they use (answer_6_2b)?

7 Append, then the snapshot



The third sitting: the cleaned extract joins the history, and notebook 14's snapshot runs on it.

7.1 The Appended Market

Make market: "source" up to "HISTORY_END", then "cleaned", with "pd.concat". "assert" its dates are unique and ascending, and that its six columns equal source's with notebook 10's "assert_frame_equal(check_exact=True)". Give its rows (answer_7_1).

7.2 The Snapshot for 2 October

Build tables with "md.snapshot_tables(market, september, '2026-10-02', units)", "units" as notebook 14's. How many tenors does the curve have (answer_7_2a)?

7.3 The Book on Four Tenors

Give the book's what-if (answer_7_3a) and DV01 in dollars per bp (answer_7_3b), to 2 places.

7.4 snapshot.py, rewritten

7.4 snapshot.py, Rewritten

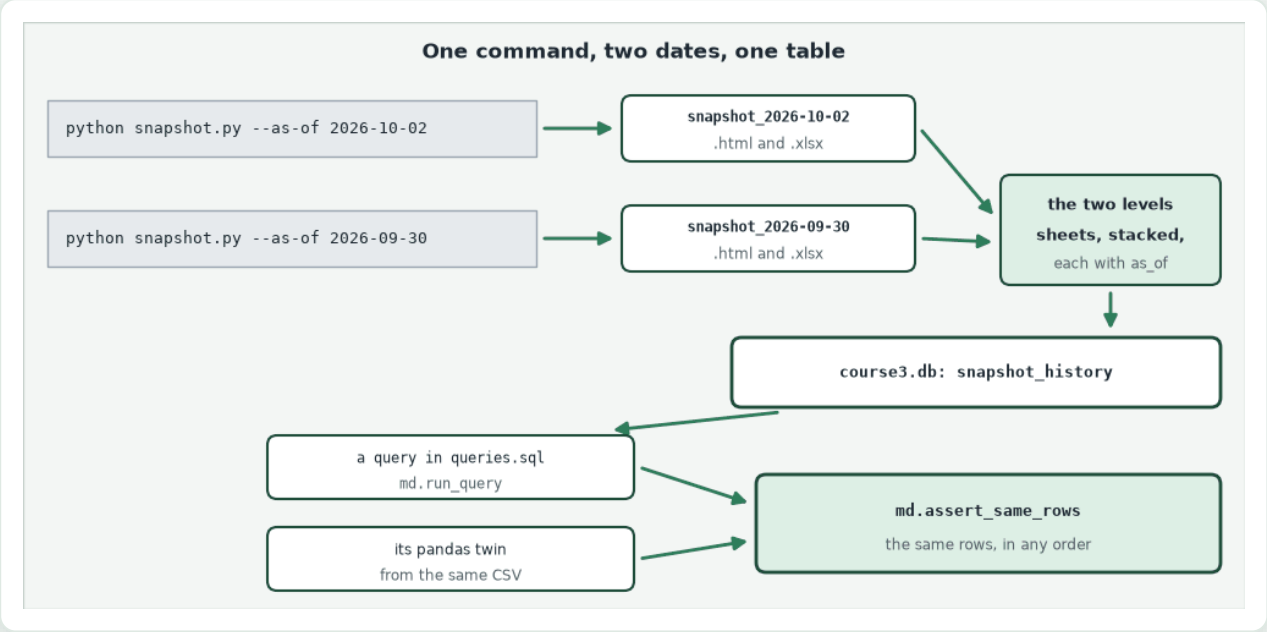
Rewrite notebook 14's "snapshot.py" with "%%writefile". "load_market()" appends the extract, cleaned by "clean_extract", to the history. "daily_snapshot(market, book, as_of)" drops each shape with a tenor empty on the date, and adds a table "notes", column "note": the curve's tenors, the shapes not shown, the what-if as a what-if, the spreads as synthetic. "main()" calls it; nothing else changes. Import it, and give the shapes it leaves out for 2 October, as a list (answer_7_4).

Read the Diff, Then Commit

Read "git diff --stat" and "git diff snapshot.py", then commit "snapshot.py".

The snapshot says what it cannot show, and the script that writes it is in the record.

8 Rerun, do not redo



8.1 Two Dates, One Command

Run "snapshot.py --as-of 2026-10-02", then "--as-of 2026-09-30", with "subprocess"; "assert" each exit code is 0. Read the 2 October page back: its date line, "md.DISCLAIMER", "synthetic", its tables. Give the 30 September workbook's sheet names, as a list (answer_8_1a), and its 10 Yr level (answer_8_1b).

Excel Side by Side

The script's workbook in desktop Excel for Windows, values with no formulas; tested there.

Job	Python	Excel
Each tenor's change on curve	`chg_bp`	`=(D2-C2)*100, filled down`
The book's DV01 and what-if from its sector rows	`the total row`	`=SUMIFS(G:G,B:B,"sector"), =SUMIFS(H:H,B:B,"sector")`

8.2 to 8.4 Side by side, the database, a test

8.2 The Two Levels Side by Side

Set the two workbooks' "levels" sheets side by side as two_dates, with a column "change_bp": 2 October less 30 September, yields times 100, spreads as they are. Which series moved most in size (answer_8_2)?

Two Dates, Six Series

Written for you: how each series moved between the two dates.

8.4 A Test, Then Commit

Write "test_capstone.py": "test_clean_extract_ties_to_the_cover_note" (66 market days, the six last values) and "test_market_equals_source" ("snapshot.load_market()" against "source"). The pytest cell below runs "test_marketdata.py" alone: add 'test_capstone.py', before it in the command, and run the cell again. Give the count passed (answer_8_4), check "course3.db" is ignored with "git check-ignore -v", and commit "queries.sql" and "test_capstone.py".

8.3 Both Days in the Database

Stack the two "levels" sheets, each row with its date as "as_of", into "snapshot_history.csv" ("as_of", "series", "unit", "level"), and load it into "course3.db" with "md.load_tables". Append two queries to "queries.sql" with "%%writefile -a": one series by a "?" parameter, and each series' change in bp with notebook 12's "LAG(level) OVER (PARTITION BY series ORDER BY as_of)" and "CASE WHEN". Check each against pandas with "md.assert_same_rows". Give the rows (answer_8_3a) and SQL's high yield change, to 1 place (answer_8_3b).

Both days are in SQL, and a test says the market equals its sources.

9 Live, with your own key (optional)

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

Your Key

```
from pathlib import Path
ENV_FILE = Path.home() / "projects" / "my_bondmath" / ".env"
NO_KEY_MESSAGE = "Live cell skipped: no FRED key was found. Every other cell runs without one."
SHOW_LICENSED_VALUES = False
```

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
```

The Four Tenors From FRED

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_ids = {'DGS2': '2 Yr', 'DGS5': '5 Yr', 'DGS10': '10 Yr', 'DGS30': '30 Yr'}
    live_curve = pd.DataFrame({live_column: md.fred_series(live_id, fred_key, start='2026-07-01',
                                                           end='2026-10-02')
                               for live_id, live_column in live_ids.items()}).dropna()
    live_common = live_curve.index.intersection(source.index)
    live_gap = (live_curve.loc[live_common] - source.loc[live_common, YIELDS]).abs().max().max()
    assert len(live_common) > 0 and live_gap <= 1e-9, 'FRED and the source files disagree'
    print(f'{len(live_common)} dates in both, largest gap {live_gap:.1e} percent')
```

A live cell, two ways

No key (the saved notebook):

```
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

The saved run had no key, so both print the skip message.

With your key:

With a key, the first prints the dates FRED and the source files share, and a gap of zero. The second prints each licensed series' rows and dates, never a value: the snapshot uses the synthetic spreads because it is shared.

10 The deliverables: the AI check, the log, the page

10.1 The AI Check

You asked an assistant for one step of "clean_extract", the docstring as the prompt. This came back, written for this course in an assistant's style: it runs, and it breaks one rule of the brief.

Find the rule it broke, fix it in its defining cell, and rerun until the test passes. Give the rows the assistant's version added that are not market days (ai_added_rows), and your version's rows (ai_rows_fixed).

10.2 The AI-Use Log

Write one row per question you asked an AI assistant; "none used" is valid. Run the cell to write "ai_use_log.md".

Nothing from an employer, no key, and no licensed data goes into an AI tool or Colab; your firm's policy comes first.

10.3 The Page

Write the page for the desk lead, 300 to 450 words, in five parts:

- the levels on both dates, with units;
- the changes to 2 October over a day, a week and a month;
- where each series sits in its year;
- the book's DV01 and what-if;
- the data: each problem, its rows, its decision.

Describe, never forecast.

Run the cell to write "description.md".

The words test, the tag, and bring it home

The Words Test

"assert_descriptive" stops on a word that forecasts or advises. The list is a reminder, not a proof: a sentence can forecast without any of them.

```
import re
FORECAST_WORDS = ['will', 'expect', 'expects', 'expected', 'should', 'cheap', 'rich', 'attractive',
                  'signal', 'buy', 'sell', 'likely', 'outlook', 'recommend', 'overweight',
                  'underweight']
def assert_descriptive(text):
    found = re.findall(r'\b(' + '|'.join(FORECAST_WORDS) + r')\b', text.lower())
    assert not found, f'words that forecast or advise: {sorted(set(found))}'
    print(f'No word from the list of {len(FORECAST_WORDS)}. Read it once more for meaning.')
assert_descriptive(Path('description.md').read_text(encoding='utf-8'))
```

Commit, Tag, and Count

Commit the log and the page together, tag the commit "course3", and show "git log --oneline --decorate". How many commits are there (answer_git)?

Bring It Home

Copy the three files and the extract into your own "my_bondmath", test, run, and commit. The new "snapshot.py" replaces notebook 14's and keeps its default date, the last in the data, so the morning task still runs.

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\clean_extract.py", "$work\snapshot.py", "$work\test_capstone.py" .
Copy-Item "$work\course3_capstone_extract.csv" .
python -m pytest -q test_marketdata.py test_capstone.py
python snapshot.py --as-of 2026-09-30
git add clean_extract.py snapshot.py test_capstone.py
git commit -m "Course 3 capstone: clean_extract, snapshot.py, and their tests"
git tag course3
```

Before You Hand In

Hand in this notebook run from the top, every check "correct", with four commits, the tag "course3", the log and the page. The planted problems are in the repo's "data/course3_capstone_answer_key.md": read it after you finish.

Predict first: the rows, and 3m10y

Predict 1

The cover note promises 66 market days. How many rows does the file hold? Write down a number.

```
raw = pd.read_csv('course3_capstone_extract.csv', dtype=str)
NOTE_DAYS = 66
```

Predict 2

The extract carries no 3 Mo yield, so after June "market" has none. On "market", "md.snapshot_tables" for 2 October shows 3m10y as:

A) no row

B) a row with no level

C) a row with June's last level, its week and month changes zero

```
HISTORY_END = '2026-06-30'
md.snapshot_tables(market, september, '2026-10-02', units)
```

Predict first: the what-if, and the assistant's rows

Predict 3

Same book, same day, and the four tenors equal the Treasury file's. Notebook 14's what-if on the whole curve was -1,193,297.89 dollars. On market's four tenors it is:

A) the same

B) smaller in size

C) larger in size

```
september = pd.read_csv('holdings.csv')
md.snapshot_tables(market, september, '2026-10-02', units)
```

Predict 4

The test data is source's 66 market days from 1 July to 2 October, with 26 August taken out: 65 rows. How many rows will the assistant's version return? Write down a number.

```
def complete_days(extract, source):
    days = pd.bdate_range(extract.index[0], extract.index[-1], name='date')
    return extract.reindex(days).ffill()

test_days = source.loc['2026-07-01':'2026-10-02', VALUE_COLUMNS]
test_input = test_days.drop(pd.Timestamp('2026-08-26'))
completed = complete_days(test_input, source)
```

The note, answered

The note asked	Answer
Prove the file	sections 1 to 4, the checkpoint
The fixes in one function	clean_extract
Nothing unexplained	the reconciliation
The snapshot for 2 October	snapshot.py, with notes
30 September, same command	8.1
Both days in the database	snapshot_history
One page	10.3

In notebook 01 your module was one function and four tests; today it is 29 functions and 55 tests, each measure with its window and units named, and each tested.

Be proud: you took a file nobody vouched for, proved every row, and handed back a page the desk can check line by line.

Next is Course 4, Machine Learning: models fitted to bond data, with the same habit of proving every number.